

1. 移动命令

beginning-of-line (C-a)

移动到当前行的开头。在某些键盘上，此功能可能绑定到 Home 键。

end-of-line (C-e)

移动到行尾。在某些键盘上，这可能也绑定到 End 键。

forward-char (C-f)

向前移动一个字符。在某些键盘上，此功能可能绑定到右箭头键。

backward-char (C-b)

后退一个字符。在某些键盘上，此功能可能绑定到左箭头键。

forward-word (M-f)

继续向前，直到下一个单词的末尾。单词由字母和数字组成。

backward-word (M-b)

返回当前单词或上一个单词的开头。单词由字母和数字组成。

shell-forward-word (M-C-f)

向前移动到下一个单词的末尾。单词之间用未加引号的 shell 元字符分隔。

shell-backward-word (M-C-b)

返回到当前单词或上一个单词的开头。单词之间用未加引号的 shell 元字符分隔。

previous-screen-line ()

尝试将光标移动到前一行屏幕上的同一物理屏幕列。如果当前 Readline 行只占用一行物理屏幕，或者光标位置小于提示符长度加上屏幕宽度，则此操作不会达到预期效果。

next-screen-line ()

尝试将指针移动到下一物理屏幕行的同一物理屏幕列。如果当前 Readline 行只占用一行物理屏幕，或者当前 Readline 行的长度不大于提示符长度加上屏幕宽度，则此操作不会达到预期效果。

clear-display (M-C-l)

清空屏幕，如果可能的话，清空终端的滚动缓冲区，然后重新绘制当前行，使当前行保持在屏幕顶部。

clear-screen (C-l)

清空屏幕，然后重新绘制当前行，使当前行保持在屏幕顶部。如果给定一个数值参数，则刷新当前行而不清空屏幕。

redraw-current-line ()

刷新当前行。默认情况下，此操作未绑定。

2. 操作历史记录的命令

accept-line (Newline or Return)

无论光标位于何处，都接受该行。如果该行非空，则根据 HISTCONTROL 和 HISTIGNORE 变量的设置将其添加到历史记录列表中。如果该行是已修改的历史记录行，则将其恢复到原始状态。

previous-history (C-p)

在历史记录列表中“后退”，获取上一个命令。在某些键盘上，此功能可能绑定到向上箭头键。

next-history (C-n)

在历史记录列表中“向前”移动，获取下一个命令。在某些键盘上，此功能可能绑定到向下箭头键。

beginning-of-history (M-<)

移至历史记录的第一行。

end-of-history (M->)

移动到输入历史记录的末尾，即当前正在输入的行。

reverse-search-history (C-r)

从当前行开始向后搜索，并根据需要向上遍历历史记录。这是一个增量搜索。此命令将区域设置为匹配的文本并激活该区域。

forward-search-history (C-s)

从当前行开始向前搜索，并根据需要向下遍历历史记录。这是一个增量搜索。此命令将区域设置为匹配的文本并激活该区域。

non-incremental-reverse-search-history (M-p)

从当前行开始，根据需要向上回溯历史记录，使用非增量搜索查找用户提供的字符串。搜索字符串可以匹配历史记录行中的任何位置。

non-incremental-forward-search-history (M-n)

从当前行开始向前搜索，并根据需要向下遍历历史记录，使用非增量搜索方式查找用户提供的字符串。搜索字符串可以匹配历史记录行中的任何位置。

history-search-backward ()

在历史记录中向前搜索当前行开头到指定位置之间的字符串。搜索字符串必须与历史记录行的开头匹配。这是一个非增量搜索。默认情况下，此命令未绑定任何按键，但在某些键盘上可以绑定到 Page Down 键。

history-search-forward ()

在历史记录中向前搜索当前行开头到指定位置之间的字符串。搜索字符串必须与历史记录行的开头匹配。这是一个非增量搜索。默认情况下，此命令未绑定任何按键，但在某些键盘上可以绑定到 Page Up 键。

history-substring-search-backward ()

在历史记录中向后搜索当前行开头到指定位置之间的字符串。搜索字符串可以匹配历史记录行中的任何位置。这是一个非增量搜索。默认情况下，此命令未绑定任何参数。

history-substring-search-forward ()

在历史记录中向前搜索当前行开头到指定位置之间的字符串。搜索字符串可以匹配历史记录行中的任何位置。这是一个非增量搜索。默认情况下，此命令未绑定任何参数。

yank-nth-arg (M-C-y)

将上一条命令的第一个参数（通常是上一行的第二个单词）插入到指定位置。如果参数为n，则插入上一条命令中的第n个单词（上一条命令中的单词从第 0 个单词开始）。如果参数为负数，则插入上一条命令末尾的第n个单词。计算出参数n后，此命令会使用历史扩展功能提取第 n个单词，就像使用 '! n历史扩展已经明确规定。

yank-last-arg (M-. or M-_)

将上一个命令的最后一个参数（上一个历史记录条目的最后一个单词）插入到该命令中。如果参数是数值，则行为与 `.` 完全相同yank-nth-arg。连续调用此命令会yank-last-arg向后遍历历史记录列表，依次插入每一行的最后一个单词（或由第一次调用的参数指定的单词）。提供给这些连续调用的任何数值参数都决定了在历史记录中移动的方向。负数参数会切换在历史记录中移动的方向（向后或向前）。这使用历史记录扩展功能来提取最后一个单词，就像使用 `!` 一样。! \$历史扩展已经明确规定。

operate-and-get-next (C-o)

接受当前行并将其返回给调用应用程序，如同输入了换行符一样；并从历史记录中获取相对于当前行的下一行进行编辑。如果提供了数值参数，则指定要使用的历史记录条目，而不是当前行。

fetch-history ()

如果传入一个数值参数，则从历史记录列表中获取该条目并将其设为当前行。如果不传入参数，则返回到历史记录列表中的第一个条目。

3. 更改文本的命令

end-of-file (usually C-d)

文件结束符由例如 表示 stty。如果读取到此字符时该行上没有字符，并且指针位于行首，则 Readline 会将其解释为输入结束并返回EOF。

delete-char (C-d)

删除指定位置的字符。如果此功能绑定到与终端结束符 (TTY EOF)相同的字符（通常情况下如此），则C-d 其效果请参见上文。在某些键盘上，此功能也可能绑定到删除键。

backward-delete-char (Rubout)

删除光标后的角色。数值参数表示杀死角色，并将其保存在击杀环中，而不是删除它们。

forward-backward-delete-char ()

删除光标下的字符，除非光标位于行尾，在这种情况下，删除光标后面的字符。默认情况下，此功能未绑定到任何按键。

quoted-insert (C-q or C-v)

C-q将输入的下一个字符原样添加到该行。例如， 这就是插入类似 这样的按键序列的方法。

self-insert (a, b, A, 1, !, ...)

输入所输入的字符。

bracketed-paste-begin ()

此功能旨在绑定到某些终端发送的“带括号的粘贴”转义序列，并且默认情况下已分配此绑定。它允许 Readline 将粘贴的文本作为一个整体插入，而无需将每个字符都视为从键盘读取的字符。字符的插入方式如同每个字符都绑定了转义序列，self-insert而不是执行任何编辑命令。

带括号的粘贴操作会将区域（句点和标记之间的字符）设置为插入的文本。它会设置活动区域。

transpose-chars (C-t)

将光标前的字符向前拖动到光标所在位置的字符上方，光标也会随之向前移动。如果插入点位于行尾，则此操作会调换该行的最后两个字符。否定参数无效。

transpose-words (M-t)

将插入点之前的单词拖动到插入点之后的单词之后，同时将插入点也移动到该单词之后。如果插入点位于行尾，则会调换该行最后两个单词的位置。

shell-transpose-words (M-C-t)

shell-forward-word将插入点之前的单词拖过插入点之后的单词，同时将插入点也移过该单词。如果插入点位于行尾，则会调换该行最后两个单词的位置。单词边界与插入点和空格 相同shell-backward-word。

upcase-word (M-u)

将当前单词（或下一个单词）转换为大写。如果参数为否定值，则将前一个单词转换为大写，但光标保持不变。

downcase-word (M-l)

将当前单词（或下一个单词）转换为小写。如果参数为否定值，则将前一个单词转换为小写，但光标保持不变。

capitalize-word (M-c)

将当前单词（或下一个单词）的首字母大写。如果参数为否定词，则将前一个单词的首字母大写，但光标保持不变。

overwrite-mode ()

切换覆盖模式。如果参数为正数，则切换到覆盖模式；如果参数为非正数，则切换到插入模式。此命令仅影响emacs覆盖模式； vi插入模式的覆盖方式不同。每次调用此命令都readline()以插入模式开始。

在覆盖模式下，绑定的字符会self-insert替换指定位置的文本，而不是将文本向右推移。绑定的字符会将backward-delete-char指定位置之前的字符替换为空格。

默认情况下，此命令未绑定，但在某些键盘上可以绑定到 Insert 键。

4. kill and yank

kill-line (C-k)

删除从光标位置到当前行末尾的文本。如果使用负数参数，则从光标位置向后删除到行首的文本。

backward-kill-line (C-x Rubout)

从光标位置向后删除到当前行开头的内容。如果参数为负数，则从光标位置向前删除到行尾的内容。

unix-line-discard (C-u)

从光标位置向后移动到当前行的开头进行删除。

kill-whole-line ()

删除当前行上的所有字符，无论光标位于何处。默认情况下，此操作未绑定任何字符。

kill-word (M-d)

从该点到当前单词的末尾，或者如果位于两个单词之间，则到下一个单词的末尾。单词边界与此相同 **forward-word**。

backward-kill-word (M-DEL)

删除句点后面的单词。单词边界与此相同 **backward-word**。

shell-kill-word (M-C-d)

从该点到当前单词的末尾，或者如果位于两个单词之间，则到下一个单词的末尾。单词边界与此相同 **shell-forward-word**。

shell-backward-kill-word ()

删除句点后面的单词。单词边界与此相同 **shell-backward-word**。

unix-word-rubout (C-w)

删除句点后面的单词，使用空格作为单词边界，并将删除的文本保存在删除环中。

unix-filename-rubout ()

删除句点后面的单词，使用空格和斜杠字符作为单词边界，并将删除的文本保存在删除环中。

delete-horizontal-space ()

删除光标周围的所有空格和制表符。默认情况下，此操作未绑定。

kill-region ()

删除当前区域中的文本。默认情况下，此命令未绑定任何字符。

copy-region-as-kill ()

将该区域中的文本复制到终止缓冲区，以便立即将其删除。默认情况下，此命令未绑定。

copy-backward-word ()

将指向终止缓冲区之前的单词复制到目标位置。单词边界与默认边界相同 **backward-word**。默认情况下，此命令不受限制。

copy-forward-word ()

将指定位置后面的单词复制到终止缓冲区。单词边界与默认边界相同 **forward-word**。默认情况下，此命令不受限制。

yank (C-y)

将杀伤环的顶部拉入缓冲区。

yank-pop (M-y)

旋转杀伤环，并拉动新的顶部。只有当先前的命令是 **yank** 或 **yank-pop** 时，才能执行此操作 **yank-pop**。

5 指定数值参数

digit-argument (M-0, M-1, ... M--)

将此数字添加到已累积的参数中，或开始一个新的参数。 **M--** 开始一个负参数。

universal-argument ()

这是指定参数的另一种方法。如果此命令后跟一个或多个数字（可选地带有前导减号），则这些数字定义参数。如果命令后跟数字，则 **universal-argument** 再次执行此命令会结束数值参数，否则此命令将被忽略。特殊情况下，如果此命令后紧跟一个既不是数字也不是减号的字符，则下一个命令的参数计数将乘以 4。参数计数初始为 1，因此第一次执行此函数时参数计数为 4，第二次执行时参数计数为 16，依此类推。默认情况下，此函数未绑定到任何键。

6. 让 Readline 为您自动输入

complete (TAB)

尝试对光标前的文本进行补全。实际执行的补全操作取决于应用程序。**Bash** 会首先检查命令词是否有任何可编程补全（参见 **可编程补全**），否则会将文本视为变量（如果文本以 **"'"** 开头）。**\$'**），用户名（如果文本以 **'** 开头）**~'**），主机名（如果文本以 **'** 开头）**@'**），或者依次尝试命令（包括别名、函数和内置命令）。如果以上方法均未找到匹配项，则回退到文件名补全。

possible-completions (M-?)

列出光标所在位置之前文本的所有可能补全项。显示补全项时，**Readline** 会按顺序 将用于显示的列

数设置为completion-display-width环境变量的值、或屏幕宽度。COLUMNS

insert-completions (M-*)

插入所有由逗号 () 生成的文本补全possible-completions, 以空格分隔。

menu-complete ()

与 `\\n` 类似complete, 但此命令会将要补全的单词替换为列表中的单个匹配项。它会重复执行menu-complete列表中的步骤, 依次插入每个匹配项。当补全列表遍历完毕后, menu-complete 它会敲响铃铛 (取决于 `\\n` 的设置bell-style), 并恢复原始文本。参数n会将匹配项在列表中向前移动nTAB个位置; 负参数则会向后移动 n 个位置。此命令原本应绑定到 `\\n`, 但默认情况下未绑定。

menu-complete-backward ()

与 `\\n` 相同menu-complete, 但会反向遍历所有可能的补全选项, 如同menu-complete传入了一个否定参数一样。此命令默认情况下未绑定任何参数。

export-completions ()

按照上述方法对句点前的单词进行补全, 并将可能的补全列表以下列格式写入 Readline 的输出流, 每行写入不同的信息:

比赛场数N;

词语完成:

S : E, 其中S和E分别是 Readline 行缓冲区中字的起始偏移量和结束偏移量; 然后

每场比赛, 每行一场

如果没有匹配项, 第一行将为“0”, 并且此命令在S : E之后不会打印任何输出。如果只有一个匹配项, 则打印包含该匹配项的单行。如果有多个匹配项, 则在 S : E之后的第一行打印匹配项的公共前缀 (该前缀可能为空), 然后在后续行打印匹配项。在这种情况下, N将包含包含公共前缀的第一行。

用户或应用程序应能处理空行的情况。其目的是让用户或应用程序读取包含“S : E”的行之后的N行, 以获取匹配列表。此命令默认情况下未绑定。

delete-char-or-list ()

如果光标所在位置不在行首或行尾 (例如 \\n), 则删除光标下的字符delete-char。如果光标位于行尾, 则其行为与 \\n 相同possible-completions。此命令默认情况下未绑定任何按键。

complete-filename (M-/)

尝试对光标前的文本进行文件名补全。

possible-filename-completions (C-x /)

列出句点之前文本的所有可能补全形式, 将其视为文件名。

complete-username (M-~)

尝试补全指针之前的文本, 将其视为用户名。

possible-username-completions (C-x ~)

列出句点之前文本的所有可能补全形式, 将其视为用户名。

complete-variable (M-\$)

尝试补全指针之前的文本, 将其视为 shell 变量。

possible-variable-completions (C-x \$)

列出句点之前文本的所有可能补全, 将其视为 shell 变量。

complete-hostname (M-@)

尝试补全指针之前的文本, 将其视为主机名。

possible-hostname-completions (C-x @)

列出 point 之前文本的所有可能补全形式, 将其视为主机名。

complete-command (M-!)

尝试对光标前的文本进行补全, 将其视为命令名称。命令补全会按以下顺序尝试将文本与别名、保留字、shell 函数、shell 内置命令以及可执行文件名进行匹配。

possible-command-completions (C-x !)

列出 `point` 之前文本的所有可能补全，将其视为命令名称。

`dynamic-complete-history (M-TAB)`

尝试补全前面的文本，将文本与历史记录列表条目进行比较，以查找可能的补全匹配项。

`dabbrev-expand ()`

尝试对光标前的文本进行菜单补全，将文本与历史记录列表中的行进行比较，以查找可能的补全匹配项。

`complete-into-braces (M-{)`

执行文件名补全，并将可能的补全列表插入到大括号中，以便 `shell` 可以使用该列表（请参阅大括号扩展）。

7. 键盘宏

`start-kbd-macro (C-x )`

开始保存当前键盘宏中输入的字符。

`end-kbd-macro (C-x )`

停止保存当前键盘宏中输入的字符，并保存定义。

`call-last-kbd-macro (C-x e)`

重新执行最后定义的键盘宏，使宏中的字符看起来像是在键盘上输入的一样。

`print-last-kbd-macro ()`

以适合 `inputrc` 文件的格式打印最后定义的键盘宏。

8. 其他一些命令

`re-read-init-file (C-x C-r)`

读取 `inputrc` 文件的内容，并合并其中找到的任何绑定或变量赋值。

`abort (C-g)`

中止当前编辑命令并按响终端的铃声（取决于设置 `bell-style`）。

`do-lowercase-version (M-A, M-B, M-x, ...)`

如果元字符 `x` 为大写，则运行绑定到相应元字符（小写）的命令。如果 `x` 已经是小写，则行为未定义。

`prefix-meta (ESC)`

将输入的下一个字符转换为元字符。正在输入“`ESCf`”等同于键入 `M-f`。

`undo (C-_ or C-x C-u)`

逐行撤销，每行单独记忆撤销操作。

`revert-line (M-r)`

撤销对此行所做的所有更改。这相当于 `undo` 多次执行该命令以恢复到初始状态。

`tilde-expand (M-&)`

对当前单词执行波浪号扩展。

`set-mark (C-@)`

将标记设置为指定点。如果提供了数值参数，则将标记设置为该位置。

`exchange-point-and-mark (C-x C-x)`

将点与标记交换位置。将当前光标位置设置为保存的位置，然后将标记设置为原来的光标位置。

`character-search (C-])`

读取一个字符并将指针移动到该字符的下一个出现位置。否定论证会查找先前出现的字符。

`character-search-backward (M-C-])`

读取一个字符并将指针移动到该字符的前一次出现位置。否定论证则搜索该字符的后续出现位置。

`skip-csi-sequence ()`

读取足够的字符以消耗多键序列，例如为 `Home` 和 `End` 键定义的序列。CSI 序列以控制序列指示符（CSI）开头，通常为 `ESC [`。如果此序列绑定到“`\el`”，则除非显式绑定到 `Readline` 命令，否则生成 CSI 序列的键将不起作用，而是将零散字符插入编辑缓冲区。默认情况下未绑定，但通常会绑

定到 ESC [。

insert-comment (M-#)

如果没有数值参数，则将变量的值插入comment-begin 到当前行的开头。如果提供了数值参数，则此命令会切换其行为：如果行首的字符与变量的值不匹配，则插入该值；否则，删除行首的comment-begin字符。无论哪种情况，该行都会被视为输入了换行符。默认值会使此命令将当前行设为 shell 注释。如果数值参数导致注释字符被删除，则该行将由 shell 执行。comment-begincomment-begin

dump-functions ()

将所有函数及其快捷键绑定打印到 Readline 输出流。如果提供了数值参数，则输出格式将便于添加到inputrc文件中。此命令默认情况下未绑定快捷键。

dump-variables ()

将所有可设置的变量及其值打印到 Readline 输出流。如果提供了数值参数，则输出格式将使其可以作为inputrc文件的一部分。此命令默认情况下未绑定任何参数。

dump-macros ()

打印所有绑定到宏的 Readline 按键序列及其输出到 Readline 输出流的字符串。如果提供了数值参数，则输出格式将便于添加到inputrc文件中。此命令默认情况下未绑定任何宏。

execute-named-command (M-x)

从输入中读取一个可绑定的 Readline 命令名称，并执行与其绑定的函数，就好像该绑定的按键序列出现在输入中一样。如果此函数接收一个数值参数，则将该参数传递给它要执行的函数。

spell-correct-word (C-x s)

对当前单词执行拼写纠错，将其视为目录或文件名，方式与cdspellshell 选项相同。单词边界与 . 使用的边界相同shell-forward-word。

glob-complete-word (M-g)

将句点前的单词视为路径名扩展的模式，隐式地附加一个星号，然后使用该模式生成匹配的文件名列表，以进行可能的补全。

glob-expand-word (C-x *)

将指针前的单词视为路径名扩展的模式，并插入匹配的文件名列表，替换该单词。如果提供了数字参数，则在末尾添加一个单引号。*' 在路径名展开之前。

glob-list-expansions (C-x g)

显示由该命令生成的扩展列表 glob-expand-word，并重新显示该行。如果提供了数值参数，则在末尾添加一个单引号。*' 在路径名展开之前。

shell-expand-line (M-C-e)

通过执行 shell 单词扩展来扩展该行。此操作会执行别名和历史记录扩展、\$'字符串' 和 \$"字符串" 引用、波浪号扩展、参数和变量扩展、算术扩展、命令和进程替换、单词分割以及引号移除。显式参数会禁止命令和进程替换。

history-expand-line (M-^)

对当前行执行历史记录扩展。

magic-space ()

对当前行执行历史记录扩展并插入一个空格（请参阅历史记录扩展）。

alias-expand-line ()

对当前行执行别名扩展（请参阅别名）。

history-and-alias-expand-line ()

对当前行执行历史记录和别名扩展。

insert-last-argument (M-. or M-_)

的同义词yank-last-arg。

edit-and-execute-command (C-x C-e)

在当前命令行调用编辑器，并将结果作为 shell 命令执行。Bash 会尝试 按顺序 调用 `<script> \$VISUAL`、`\$EDITOR`<script>` 和`<script>` 作为编辑器。emacs

`display-shell-version (C-x C-v)`
显示当前 Bash 实例的版本信息。